SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	Vasantha Nithi D Y	**Reg. No.:**	192424137
Section / Batch:	2024	**Date:**	

Code of Conduct

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
For the given student details formulate Regex patterns for the following:
Email ID
Mobile Number
Strong Password
Date of Birth (DD/MM/YYYY)
Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.
Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.
Validation Criteria:
Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, $, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1	3	3	4	4	4	18	86
2	4	4	3	4	4	19	
3	3	4	4	3	4	18	
4	4	3	4	4	3	18	
5	3	4	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Question 1 – Regular Expression Pattern Generation

Problem Statement

For the given student details, formulate Regular Expression (Regex) patterns to validate the following fields:

- Email ID
- Mobile Number
- Strong Password
- Date of Birth (DD/MM/YYYY)
- Register Number

Write a Python program using the **re** module to validate each field.

Regex Patterns

Field	Regex Pattern
Email ID	^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
Mobile Number	^\+91-\d{10}$
Strong Password	^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$
Date of Birth	^\d{2}/\d{2}/\d{4}$
Register Number	^\d{2}[A-Z]{4}\d{4}$

Pattern Description

Email ID

Matches a valid email address containing a username, **@** symbol, domain name, and domain extension.

Mobile Number

Matches an Indian mobile number beginning with **+91-** followed by exactly **10 digits**.

Strong Password

Matches a password that:

- contains at least 8 characters,
- has at least one uppercase letter,

- has at least one lowercase letter,
- has at least one digit,
- has at least one special character (@ # $ % & ! *).

Date of Birth

Matches dates in the format **DD/MM/YYYY**.

Register Number

Matches a register number consisting of:

- 2 digits (Year)
- 4 uppercase letters (Department Code)
- 4 digits (Roll Number)

Example: **23AIML1056**

Algorithm

1. Import the re module.
2. Store the student details.
3. Define the regex pattern for each field.
4. Use re.match() to compare each value with its corresponding pattern.
5. Display whether each value is **Valid** or **Invalid**.

Python Program

```
import re


student = {
    "Email": "john.doe123@gmail.com",
    "Mobile": "+91-9876543210",
    "Password": "P@ssw0rd123",
    "DOB": "15/08/2004",
    "Register": "23AIML1056"
}


patterns = {
```

```
    "Email": r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$",
    "Mobile": r"^\+91-\d{10}$",
    "Password": r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$",
    "DOB": r"^\d{2}/\d{2}/\d{4}$",
    "Register": r"^\d{2}[A-Z]{4}\d{4}$"
}

for field, pattern in patterns.items():
    if re.match(pattern, student[field]):
        print(f"{field}: Valid")
    else:
        print(f"{field}: Invalid")
```

Output

Email: Valid

Mobile: Valid

Password: Valid

DOB: Valid

Register: Valid

Question 2 – Search for a String

Problem Statement

Write a Python program using the **re** module to search for the word **"AI"** in the given passage. Display the first occurrence, starting position, ending position, total number of occurrences, and print an appropriate message if the word is not found.

Regex Pattern

\bAI\b

Description: Matches the complete word **"AI"** using word boundaries.

Algorithm

1. Import the re module.
2. Store the given passage in a string.
3. Define the regex pattern \bAI\b.
4. Use re.search() to find the first occurrence.
5. Display the starting and ending positions using start() and end().
6. Use re.findall() to count all occurrences.
7. If no match is found, display an appropriate message.

Python Program

```
import re


text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""


pattern = r"\bAI\b"


match = re.search(pattern, text)


if match:
    print("First occurrence found.")
    print("Starting Position:", match.start())
    print("Ending Position:", match.end())
    print("Total Occurrences:", len(re.findall(pattern, text)))
else:
    print("Word 'AI' not found.")
```

Output

First occurrence found.

Starting Position: 25

Ending Position: 27

Total Occurrences: 6

Question 3 – Split the Documentation into Sentences and Words

Problem Statement

Write a Python program using the **re.split()** method to perform the following operations on the given passage:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count the total number of sentences.
- Split the passage into words using one or more whitespace characters as delimiters.
- Count the total number of words.
- Display the list of extracted sentences and words.

Regex Patterns

Sentence Split

[.!?]+

Word Split

\s+

Pattern Description

Sentence Split ([.!?]+)

- Splits the text wherever a period (.), question mark (?), or exclamation mark (!) appears.
- The + operator allows one or more punctuation marks to be treated as a single delimiter.

Word Split (\s+)

- Splits the text using one or more whitespace characters such as spaces, tabs, or new lines.

Algorithm

1. Import the re module.
2. Store the given passage in a string.
3. Use re.split(r'[.!?]+', text) to split the passage into sentences.
4. Remove empty sentences from the list.
5. Use re.split(r'\s+', text) to split the passage into words.
6. Count the number of sentences and words.
7. Display the extracted sentences, extracted words, and their counts.

Python Program

```
import re


text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""


# Split into sentences

sentences = re.split(r'[.!?]+', text)

sentences = [s.strip() for s in sentences if s.strip()]


# Split into words

words = re.split(r'\s+', text.strip())


print("Extracted Sentences:")

for s in sentences:

    print(s)


print("\nTotal Sentences:", len(sentences))


print("\nExtracted Words:")
```

```
print(words)


print("\nTotal Words:", len(words))
```

Output

Extracted Sentences:

Artificial Intelligence (AI) is transforming industries across the world

AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences

Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making

As AI continues to evolve, professionals with AI skills are in high demand

Total Sentences: 4

Extracted Words:

['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', ..., 'demand.']

Total Words: 54

Result

The given passage was successfully split into individual sentences and words using the re.split() method. The program displayed the extracted sentences, extracted words, and their respective counts, demonstrating text tokenization using Python Regular Expressions.

Question 4 – Email and Strong Password Validation

Problem Statement

A website requires users to register using an **Email ID** and a **Strong Password**. Write a Python program using the **re** module to validate the given Email ID and Strong Password based on the specified validation criteria.

Regex Patterns

Email ID

^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$

Strong Password

^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])\S{8,}$

Pattern Description

Email ID

- Begins with one or more letters or digits.
- May contain ., _, %, +, or - before @.
- Contains a valid domain name.
- Ends with a valid domain extension (.com, .org, .edu, or .in).

Strong Password

- Contains at least 8 characters.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one digit.
- Contains at least one special character (@, #, $, %, &, !, *).
- Does not contain whitespace.

Algorithm

1. Import the re module.
2. Read the Email ID and Password.
3. Define the regex patterns for Email ID and Password.
4. Use re.fullmatch() to validate both inputs.
5. Display whether the Email ID and Password are valid or invalid.

Python Program

```
import re


email = "john.doe123@gmail.com"

password = "P@ssw0rd123"
```

```
email_pattern = r"^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$"
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])\S{8,}$"

if re.fullmatch(email_pattern, email):
    print("Email ID: Valid")
else:
    print("Email ID: Invalid")

if re.fullmatch(password_pattern, password):
    print("Password: Valid")
else:
    print("Password: Invalid")
```

Output

Email ID: Valid

Password: Valid

Result

The program successfully validated the given Email ID and Strong Password using Python's re module. Both inputs satisfied the specified validation criteria and were identified as **Valid**.

Question 5 – Compile Once, Validate Many

Problem Statement

A company receives **1,000 Email IDs** from its customers and needs to validate each one efficiently. Instead of compiling the same Regular Expression repeatedly, write a Python program using the **re.compile()** method to compile the pattern once and reuse it to validate all Email IDs.

Regex Pattern

^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$

Pattern Description

- Begins with one or more letters or digits.
- Allows ., _, %, +, and - before the @ symbol.
- Contains a valid domain name.
- Ends with one of the valid domain extensions: **.com**, **.org**, **.edu**, or **.in**.
- The pattern is compiled once using re.compile() and reused for validating multiple Email IDs, improving performance.

Algorithm

1. Import the re module.
2. Compile the Email ID regex pattern using re.compile().
3. Store the Email IDs in a list.
4. Iterate through each Email ID.
5. Use the compiled pattern to validate each Email ID.
6. Display whether each Email ID is valid or invalid.

Python Program

```
import re


# Compile the regex pattern once
email_pattern = re.compile(
    r"^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$"
)


emails = [
    "john.doe123@gmail.com",
    "student@saveetha.edu",
    "user123@yahoo.in",
    "invalid@email",
    "@gmail.com"
]
```

```
for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

Output

```
john.doe123@gmail.com -> Valid
student@saveetha.edu -> Valid
user123@yahoo.in -> Valid
invalid@email -> Invalid
@gmail.com -> Invalid
```

Result

The Email ID validation pattern was compiled once using the **re.compile()** method and successfully reused to validate multiple Email IDs. This approach improves efficiency and performance when validating a large number of inputs.